DEVOPS MINI PROJECT — INTERNAL DOCUMENTATION

Exxomm — Automated CI/CD Pipeline on AWS

This document walks through what I actually built: the infrastructure, the pipeline that deploys on top of it, and the reasoning behind the decisions I had to make along the way — some of them because of how the requirements were written, a couple because of account-level constraints I ran into partway through.

Author: Gopi Kumar · Region: ap-south-1 · Last reviewed: November 2026

---

01 — OVERVIEW

What this project is, and why it's built this way

The brief was simple to state and a bit more involved to actually satisfy: provision AWS infrastructure with Terraform, and deploy a web application onto it automatically with Jenkins, such that a single commit is enough to trigger the right work — nobody should ever need to click around the AWS console or SSH into a box to get a change live. That last part is the success criterion the whole design is judged against, and it shaped almost every decision below, including a few that aren't obvious until you see what they're protecting against.

Exxomm itself is a small MERN e-commerce storefront — a React frontend, a Node/Express API, and a MongoDB database. None of that is the interesting part of this project. The interesting part is everything around it: how the infrastructure is described in code, how it's provisioned safely, and how a pipeline decides what to run depending on what actually changed in a given commit.

Technology stack

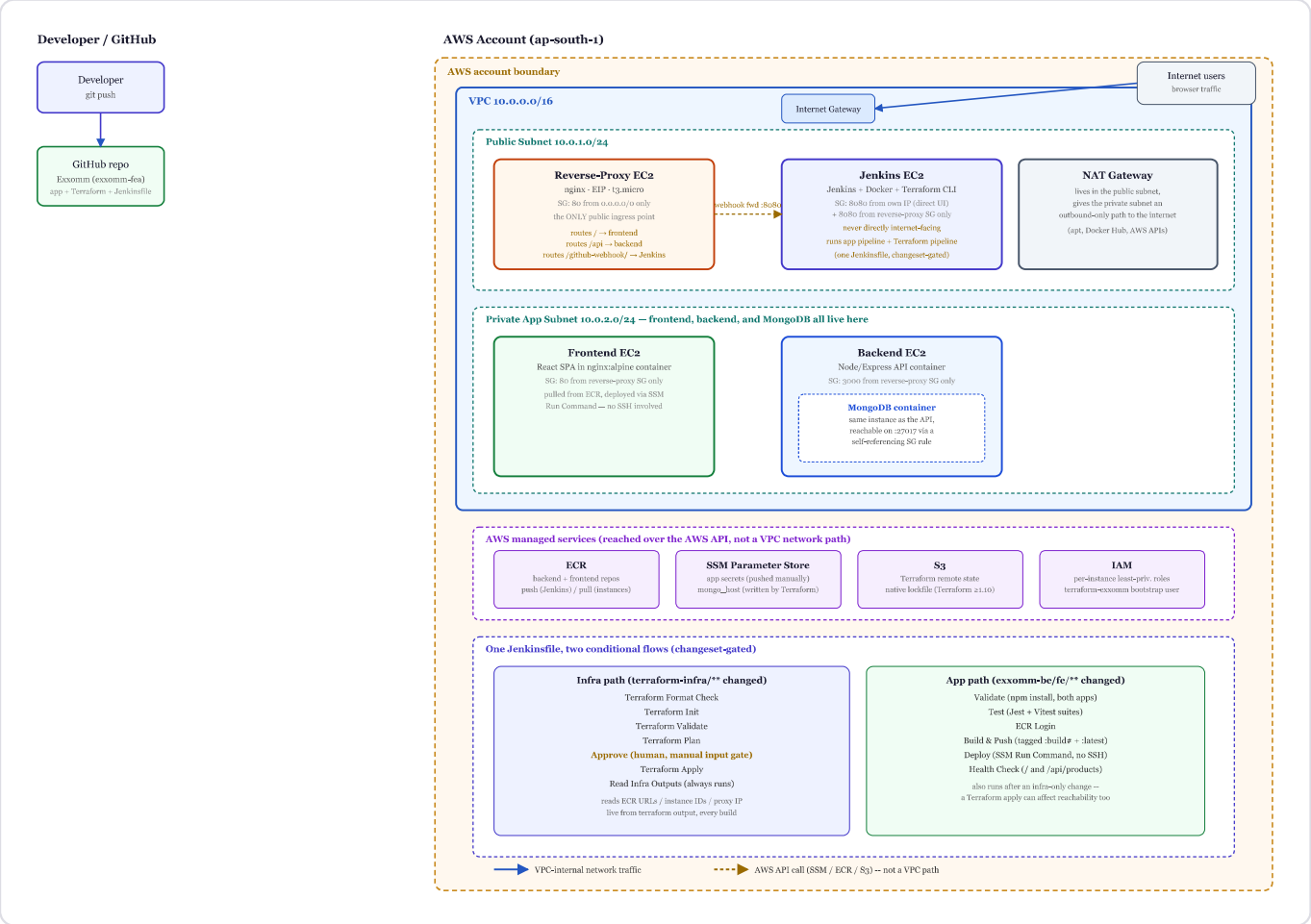
LAYER	TOOL	WHY THIS ONE
Infrastructure	Terraform ≥1.10	Declarative, and the project explicitly required a reusable networking module rather than hand-wired resources
Remote state	Amazon S3	Both my laptop and Jenkins need to see the same state — local state only works for one person at a time
CI/CD	Jenkins	Declarative Pipeline syntax, native `changeset` conditions, and runs happily on a small EC2 instance
Containers	Docker	Backend, frontend, and the MongoDB container all ship as images rather than hand-installed services
Gateway	nginx	One reverse proxy in front of everything, and it also happens to forward GitHub's webhook internally to Jenkins
Registry	Amazon ECR	Private, region-local, and the instance IAM roles can pull from it without any stored credentials
Secrets	SSM Parameter Store	Cheaper and simpler than Secrets Manager for a project this size, and it satisfies the "store secrets via IAM roles" requirement directly
Operator access	SSM Session Manager / Run Command	No SSH key to lose, no bastion host to maintain, no port 22 open anywhere

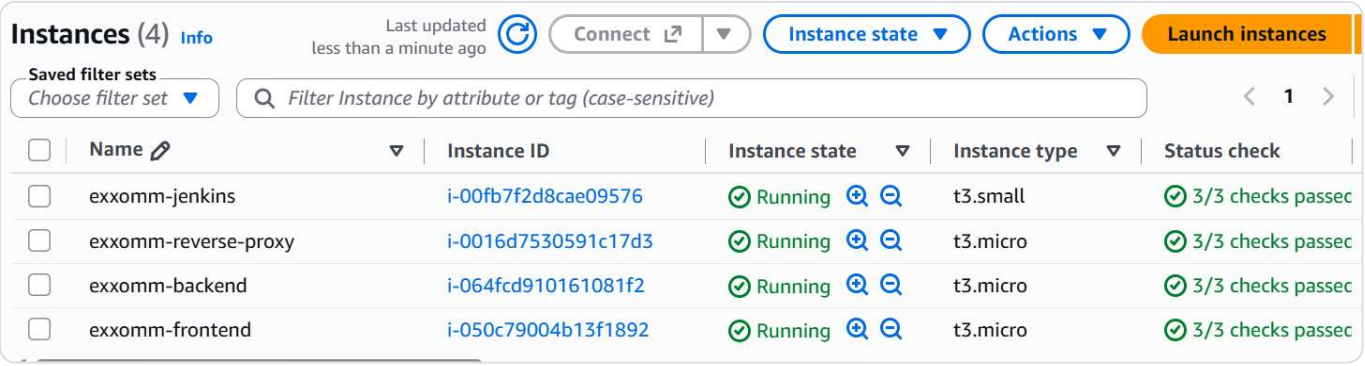
If there's one idea to take away from this whole document, it's that the infrastructure and the deployment pipeline were kept deliberately separate. Terraform provisions things once; Jenkins only ever builds, tests, and deploys the application on top of whatever already exists. The two do meet inside one Jenkinsfile now (more on that in the Workflows section), but even there, each one only runs when the commit actually touches its part of the repository.

02 — ARCHITECTURE

Network layout

Everything lives in a single-AZ VPC (`ap-south-1a` — there was no budget, literal or otherwise, for multi-AZ redundancy on this pass). One EC2 instance, the reverse proxy, is the only thing the public internet can reach. Every other instance only accepts traffic from the one security group that's actually allowed to talk to it — nothing is reachable "just because it's in AWS," which is the whole point of calling this a zero-trust ingress model rather than just a private subnet with a firewall.





The four EC2 instances as they actually appear in the console — one box per role, matching the diagram above exactly. Only four instances exist; there's no fifth box for Mongo because it isn't a separate instance, it's a container running on `exxomm-backend`.

One thing that's easy to miss looking at a diagram like this: the private app subnet has no route to the internet gateway at all, only to the NAT gateway. That's a one-way door — instances inside can initiate outbound connections (to pull a Docker image, say, or read a secret), but nothing from the internet can initiate a connection in. Combined with the security-group rules, this is really two independent layers doing the same job from different directions, which is deliberate rather than redundant.

03 — NETWORKING & IAM

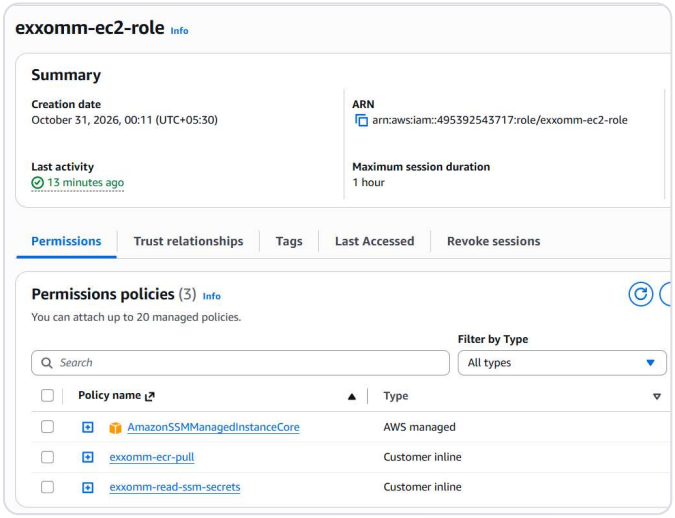
The VPC module, and who's allowed to do what

VPC and subnets

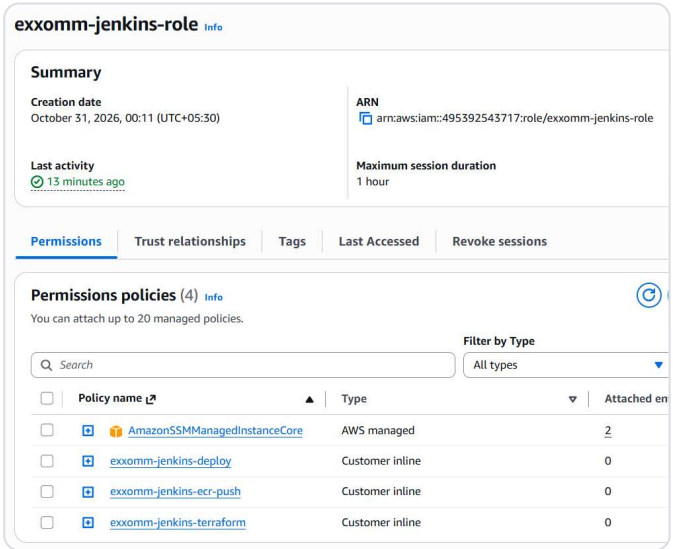
The project requirements were specific about this one: infrastructure had to be built with Terraform, and the networking piece specifically had to be a reusable module rather than inline resources — so `modules/networking` owns the VPC, both subnets, the internet gateway, the NAT gateway, and the route tables, and the root configuration just calls it with a handful of CIDR variables. The VPC itself is `10.0.0.0/16`, split into a public subnet (`10.0.1.0/24`, where the reverse proxy, Jenkins, the NAT gateway, and the internet gateway attachment live) and a private application subnet (`10.0.2.0/24`, where the frontend, the backend, and the MongoDB container all sit). There's deliberately no third subnet for data — an earlier pass of this design did split out a private data subnet for a dedicated Mongo instance, but once Mongo moved onto the backend instance as a container (see the Limitations section for why), keeping an empty subnet around just to represent a future possibility stopped making sense, so it was removed rather than left in as unused scaffolding.

IAM roles — one per job, nothing shared

Every EC2 instance in this project assumes an IAM role rather than carrying stored AWS credentials, and the roles are split by what the instance actually needs to do, not grouped by convenience. There are two roles that matter:



`exxomm-ec2-role` — shared by frontend, backend, and (while Mongo is colocated) effectively the database too. Three policies: the AWS-managed SSM core policy for Session Manager access, a scoped ECR-pull policy, and a scoped SSM-parameter-read policy limited to `/exxomm/prod/*`.



`exxomm-jenkins-role` — SSM core for Session Manager, an ECR-push policy, an SSM Run Command / parameter-write policy scoped to the two deploy targets, and `exxomm-jenkins-terraform` — the policy that lets Jenkins run

`terraform plan` / `apply` itself once that pipeline flow was added.

Neither of these roles can do anything outside its own lane. The EC2 role can read secrets and pull images, full stop — it has no permission to touch IAM, networking, or anything Terraform-shaped. The Jenkins role is the more powerful of the two by necessity, since it now runs Terraform on my behalf, but even there the IAM and EC2 permissions it holds are scoped to resource names starting with `exxomm-`, so it can't reach into any other part of the AWS account.

There's a third identity worth mentioning that doesn't show up in Terraform at all: `terraform-exxomm`, a plain IAM user I created by hand, with access keys configured locally, used only to run Terraform from my own machine before Jenkins could do it itself. It's scoped the same way the Jenkins role is — least privilege, resource names restricted to this project — and I've gotten in the habit of deactivating its access key rather than leaving it live, since there's no reason for a long-lived credential like that to be usable every day once the pipeline can do the same job.

04 — WORKFLOWS

One Jenkinsfile, two conditional flows

I went back and forth on this one. The cleaner production pattern is genuinely two separate pipelines — a rarely-run infrastructure pipeline and a frequently-run application pipeline, each with its own trigger and its own blast radius. But for a single-repository, single-pipeline project like this, splitting it in two mostly adds operational overhead without adding much safety, so I settled on one Jenkinsfile with two conditional flows instead, gated by Jenkins' `changeset` condition. A commit that only touches application code never runs a Terraform stage; a commit that only touches `terraform-infra/` never rebuilds or redeploys the app. The tradeoff is explained properly in the Limitations section, because it is a real tradeoff, not a free lunch.

Infra flow — runs when `terraform-infra/**` changed

Checkout → Format Check → Init → Validate → Plan → Approve (human) → Apply → Read Infra Outputs

The approval gate is a plain Jenkins `input` step — the plan is printed to the console, the build sits there waiting, and a person has to read it and click Approve or Abort. I added this after enough near-misses while building this project by hand (a typo'd CIDR, an instance I didn't mean to replace) to be unwilling to let a plan apply itself unreviewed, even though it would have been a few lines simpler without it.

App flow — runs when `exxomm-be/**` or `exxomm-fe/**` changed

Checkout → Validate → Test → ECR Login → Build & Push → Deploy (SSM) → Health Check

Validate is really just `npm install` for both apps — neither has a lint script yet, so this stage exists mostly to catch a broken lockfile before spending time on anything else. Test runs the real suites (Jest for the backend, Vitest for the frontend) against source, before any image is built, specifically so a failing test can never end up inside a pushed image. Deploy never touches SSH — it ships a small shell script to the target instance over AWS SSM Run Command, and that instance's own IAM role does the actual ECR pull and secret lookup. Health Check is the final word: it hits the live reverse-proxy URL for both `/` and `/api/products` and fails the build outright if either doesn't come back with a 200.



The Stage View for the `exxomm-cicd` job across several real builds. The red blocks are from two builds I broke on purpose while getting the Terraform-apply permissions right on Jenkins' own IAM role — left in here rather than cropped out, because a documentation page that only ever shows green builds isn't telling the whole story.

Triggering a build

A build can start two ways: a GitHub push (through the webhook, covered in the next section) or a manual "Build with Parameters" click. Either way, the job takes three parameters, shown below as they actually appear in the Jenkins UI.



Everything that used to be a hand-typed parameter — ECR repository URLs, instance IDs, the reverse-proxy address — is gone now. The `Read Infra Outputs` stage pulls those straight out of `terraform output` on every run, so what's left to configure is genuinely just the region, the admin CIDR for direct Jenkins access, and an override to force every stage to run on a build Jenkins has no changelog for yet.

> A change to the Jenkinsfile itself runs both flows in full, regardless of `changeset`. A pipeline-logic edit can affect any stage's behaviour, and I'd rather a Jenkinsfile-only commit over-run everything once than silently report success having run nothing which is exactly what happened the first time I forgot this rule.

05 — REQUEST FLOWS

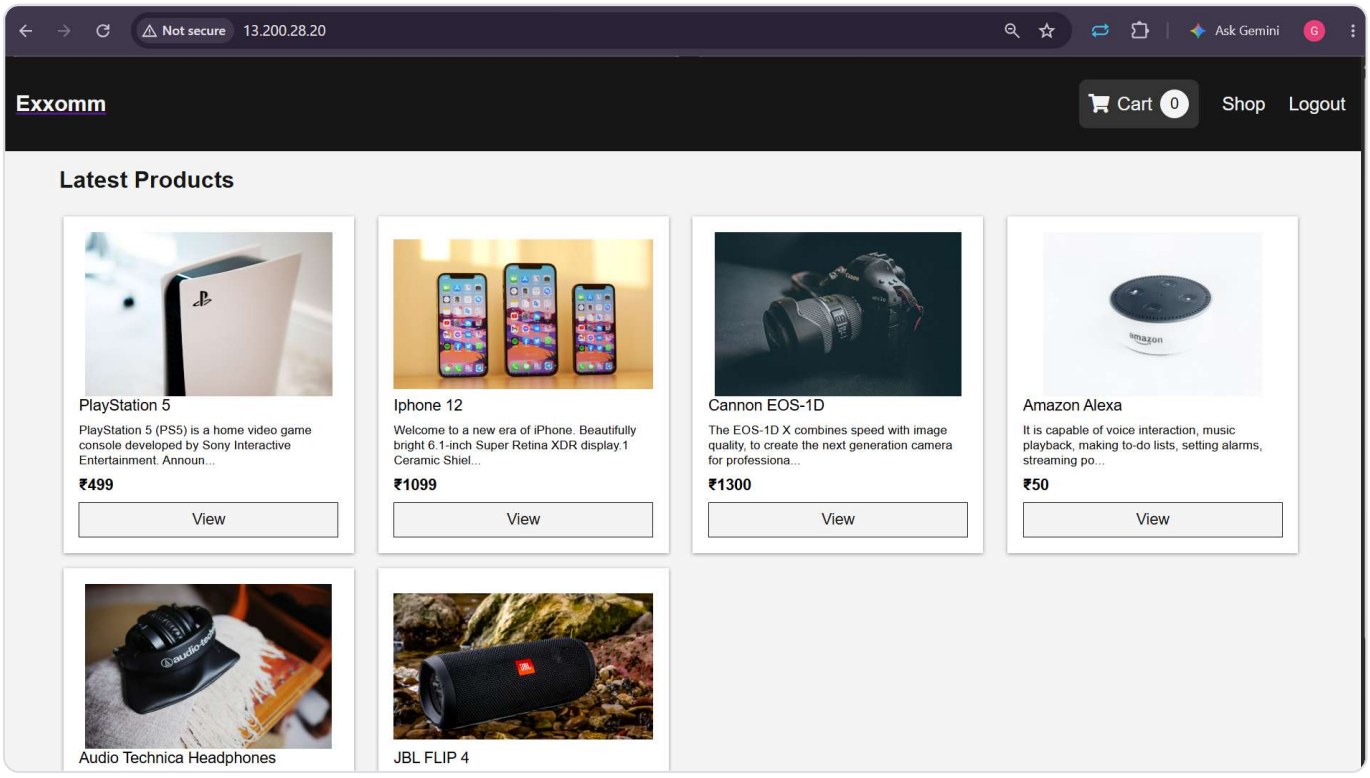
How a browser request — or a commit — actually moves

End-user request flow

Browser → Reverse-Proxy (nginx, :80) → Frontend SPA (/)

Browser → Reverse-Proxy (nginx, :80) → Backend API (/api) → MongoDB (:27017)

The browser only ever knows about one address — the reverse proxy's public IP. It has no idea the frontend and backend even exist as separate machines, and it has no way to find out, since neither one accepts a connection from anywhere except the proxy's own security group.



The storefront, loaded by hitting the reverse proxy's public IP directly — this is the response that proves the whole chain (proxy → frontend container → browser) actually works, not just that the pipeline reported success.

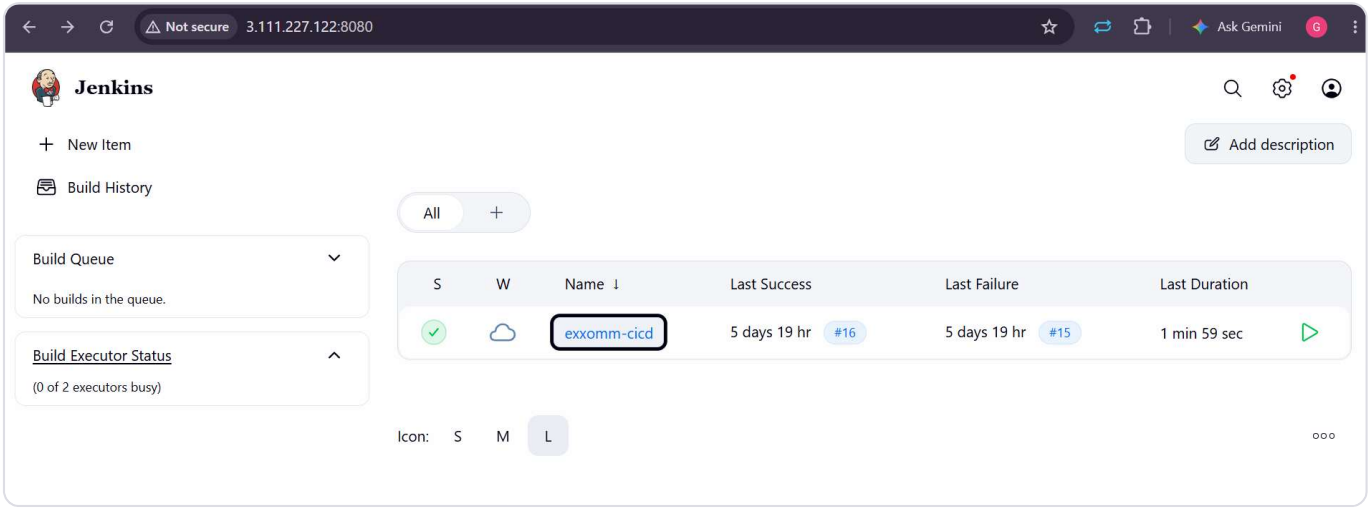
Commit-to-deployed-app flow

Developer push → GitHub webhook → Reverse-Proxy forwards → Jenkins triggers build →

Infra or App flow runs → Live app updated

The webhook URL I gave GitHub points at the reverse proxy, not at Jenkins directly — `http://<reverse-proxy-ip>/github-webhook/`. nginx recognises that one path and quietly forwards it

over the private network to Jenkins on port 8080. I built this specifically so Jenkins' own security group would never need a rule opening it to the whole internet just to receive webhook traffic; the reverse proxy was already public, so the webhook rides along the same door everything else already uses.



The job as it sits in the Jenkins dashboard — last success, last failure, and the build history that a webhook-triggered run quietly adds to without anyone opening the Jenkins UI at all.

06 — DEPLOYMENT

Setting it up from scratch

This is the part I'd actually hand someone if they needed to rebuild this from nothing. It's written in the order the steps genuinely have to happen in, which isn't always the order you'd guess — pushing secrets before the first `terraform apply`, for instance, matters more than it looks like it should.

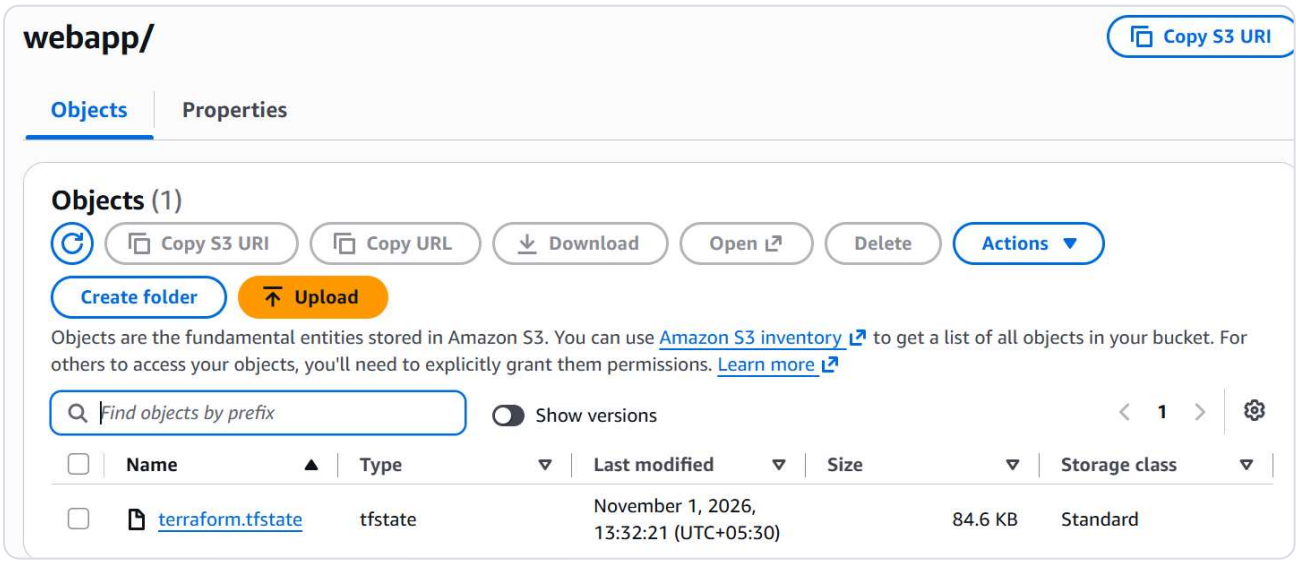
1 Create the bootstrap IAM identity

Terraform has to be run by something before any of this exists, and that something shouldn't be a personal admin account. `terraform-exxomm` is a plain IAM user scoped to exactly what this project provisions: `ec2:*`, ECR and SSM actions, S3-state access, and IAM role/instance-profile management restricted to resource names starting with `exxomm-`, with `iam:PassRole` further restricted so it can only hand those roles to `ec2.amazonaws.com`.

2 Create and secure the S3 state bucket

```
aws s3api create-bucket --bucket <bucket> --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1
aws s3api put-bucket-versioning --bucket <bucket> --versioning-configuration Status=Enabled
aws s3api put-bucket-encryption --bucket <bucket> --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'
aws s3api put-public-access-block --bucket <bucket> --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=tr
```

The backend block already lives in `terraform-infra/main.tf`; native S3 locking (`use_lockfile = true`) needs Terraform 1.10 or newer, which is what let me skip standing up a DynamoDB lock table entirely.

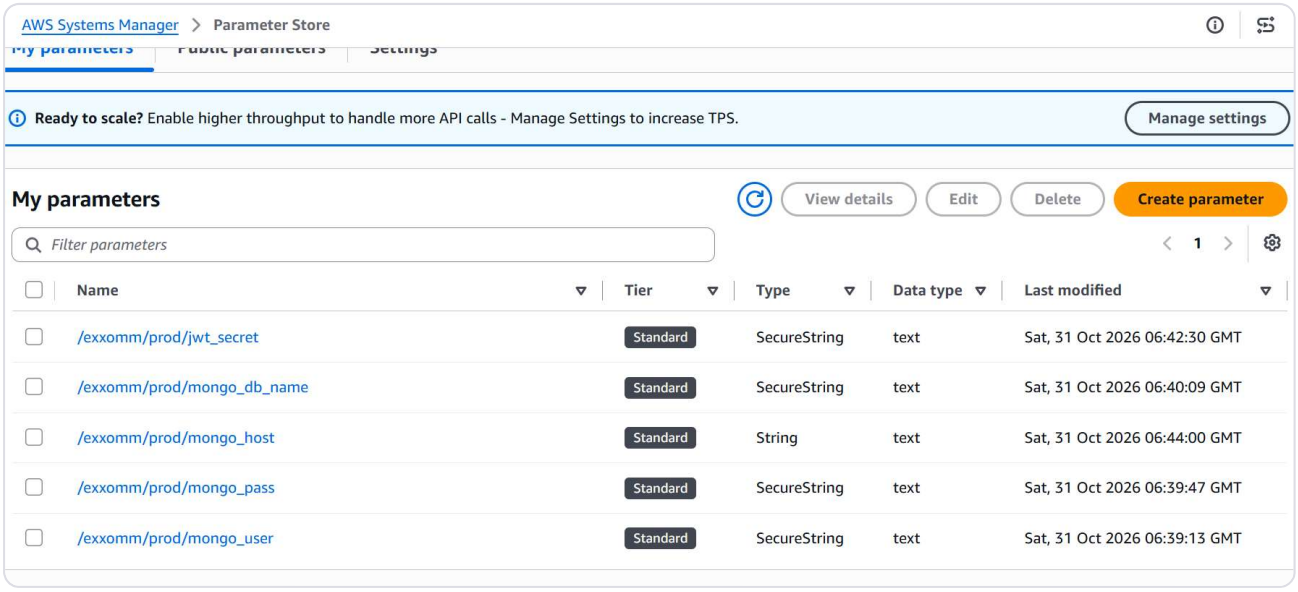


The bucket once it's actually in use — one object, `terraform.tfstate`, 84.6 KB at the time of this screenshot. Both my laptop and Jenkins read and write this same file now, which is the entire point of moving off local state.

3 Push the application secrets to SSM — before the first apply, not after

```
aws ssm put-parameter --region ap-south-1 --type SecureString --name /exxomm/prod/mongo_user
aws ssm put-parameter --region ap-south-1 --type SecureString --name /exxomm/prod/mongo_pass
aws ssm put-parameter --region ap-south-1 --type SecureString --name /exxomm/prod/mongo_db_r
aws ssm put-parameter --region ap-south-1 --type SecureString --name /exxomm/prod/jwt_secret
```

> I got this ordering wrong once already, and it's worth flagging why it matters: the backend instance's boot script reads `mongo_user` / `mongo_pass` / `mongo_db_name` from SSM the moment it starts, to bring the Mongo container up immediately. If those parameters don't exist yet, that boot step just fails quietly — Terraform itself reports success, because from Terraform's point of view the instance launched fine. `mongo_host` is the one exception; Terraform writes that one itself, since it's just the instance's own private IP, not a secret.



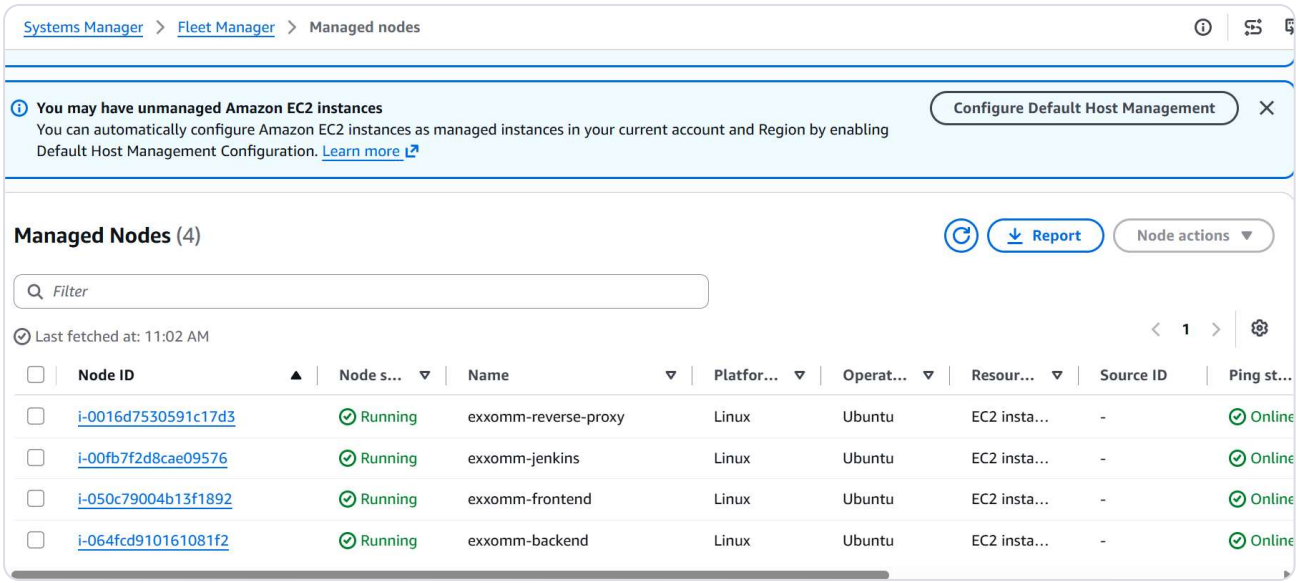
Parameter Store once everything is in place — four `SecureString` values I pushed by hand, and `mongo_host` sitting there as a plain `String`, the only one of the five Terraform ever touches.

4 Apply the infrastructure

```
cd terraform-infra
terraform init
terraform apply -var="jenkins_admin_cidr=<your-ip>/32"
```

From here, every instance is reachable only through SSM — there's no SSH key anywhere in this project. Fleet Manager shows all four instances online the moment their boot scripts finish installing the SSM

agent.



All four instances registered with Systems Manager and reporting `Online` — this is what "no SSH anywhere" looks like in practice: every box is reachable for debugging or a manual fix, just never over port 22.

5 Point the Jenkins job at the repository

Install the Pipeline and Git plugins, create a Pipeline job, and set its Script Path to `jenkins/Jenkinsfile` rather than the repository root. The only parameters left to fill in are the AWS region, the admin CIDR for direct Jenkins access, and the first-run override mentioned earlier.

6 Wire up the GitHub webhook — through the reverse proxy

```
http://<reverse-proxy-public-ip>/github-webhook/
```

Content type `application/json`, "Just the push event," and **GitHub hook trigger for GITScm polling** enabled on the job itself. This is the piece that actually satisfies the success criterion — from here on, a commit is enough.

Rolling back

`:latest` in ECR only ever moves after a successful push, and every build is also tagged with its own build number, so an older image is never actually gone. Rolling back means re-running the deploy script by hand with that older tag:

```
jenkins/scripts/deploy-via-ssm.sh <BACKEND_INSTANCE_ID> jenkins/scripts/deploy-backend.sh <BACKEN
```

07 — LIMITATIONS

Where this falls short of a real production setup, and why I let it

None of the following are things I didn't notice. They're tradeoffs I made on purpose, usually because of either the scope of the actual assignment or a constraint I ran into partway through that wasn't worth fighting at this stage. I'd rather write them down than have someone assume I didn't think about them.

One Jenkinsfile with two `changeset`-gated flows, instead of two separate pipelines. This is simpler to operate for a single-pipeline project, but it has a real cost: a commit that touches both infrastructure and application code runs both flows in the same build, with only one approval gate between them, and a failure partway through leaves a messier recovery story than two independently-schedulable pipelines would. If this project grew past a mini-project, splitting it would be the first thing I'd do.

Plain Docker containers on long-lived EC2 instances, deployed over SSM Run Command — not a real orchestrator. This matches what was actually asked for, but it means no auto-scaling, no self-healing if a container or instance crashes, and no rolling or blue-green deploys. Container lifecycle (stop, remove, run) is hand-rolled in shell scripts rather than handled by a scheduler. ECS or EKS is the production-grade equivalent.

MongoDB runs as a container on the backend instance instead of its own dedicated EC2. This is the one limitation that actually changed shape partway through the project. The original design had a separate Mongo instance in its own subnet, exactly like a real production data tier should be isolated — but this AWS account's "Running On-Demand Standard" vCPU quota is capped at 8, and five always-on instances (including Jenkins) didn't fit under it. Rather than wait on a quota increase that was still pending, I moved Mongo onto the backend instance as a second container, reachable only through a security-group rule that allows the backend's own SG to reach itself on port 27017. The dedicated-instance version is still fully defined in Terraform — it's just set to `count = 0` behind a `colocate_db_with_backend` variable — so reverting it once the quota increase clears is a one-line change, not a rebuild.

No TLS or HTTPS anywhere. The reverse proxy serves plain HTTP on port 80 only — there's no domain name and no certificate provisioned, so every request, including a login submission, is unencrypted in transit. This was explicitly out of scope for this pass rather than an oversight; adding it later just means acquiring a domain and an ACM certificate and updating the proxy's nginx config to terminate TLS.

Egress security-group rules are broad, while ingress is tightly scoped. Every instance can reach anywhere on any port outbound, which sits oddly next to how carefully the inbound rules are written. The honest reason is that AWS security groups can only restrict egress to a CIDR, another security group, or an AWS-managed prefix list — none of which can represent "only Docker Hub" or "only GitHub," since those are internet services on large, rotating IP ranges with no stable range to allowlist. Scoping egress tightly for AWS's own services (ECR, S3, SSM) is

possible with VPC Endpoints, which I didn't set up here — a reasonable next step, at the cost of a few new resources and a small ongoing per-endpoint charge.

The reverse proxy's nginx config is written once, at boot, from the other instances' private IPs at the time Terraform applies. If any of those instances is ever replaced outright — not just redeployed in place, which is all this pipeline normally does — the proxy doesn't pick up the new IP on its own. It needs a forced replace of its own to re-render that config. Given the current scope (one long-lived instance per role, no auto-scaling), I accepted this rather than building a more dynamic service-discovery mechanism for a project this size.
